

Task Manager – Day 3 Progress

Simple explanation of what was built and tested

1. What was the main goal of Day 3?

The main goal was to add **Tasks** to the Task Manager and allow a logged-in user to create, view, update and delete only their own tasks.

Simple example: Hitesh logs in and creates a task called “Learn Spring Boot”. That task belongs to Hitesh. Another user should not be able to see or change it.

2. Task structure

A Task contains the following information:

Field	Simple meaning	Example
id	Unique number for the task	1
title	Task name	Learn Spring Boot
description	More details about the task	Complete Task Manager project
status	Current progress	TODO / IN_PROGRESS / DONE
dueDate	Date by which the task should be completed	2026-10-10
user	The user who owns the task	hiteshtest@gmail.com

3. TaskStatus enum

We created three possible task statuses:

TODO – task has not started.

IN_PROGRESS – task is currently being worked on.

DONE – task has been completed.

Example: A task starts as TODO → while working, it becomes IN_PROGRESS → after finishing, it becomes DONE.

4. Repository – talking to the database

TaskRepository was created to communicate with the database through Spring Data JPA.

We added methods to find all tasks belonging to a particular user and to find a specific task only when it belongs to that user.

Why this matters: Instead of simply asking for task ID 1, the application asks for “task ID 1 belonging to this logged-in user”.

5. DTO – controlling API data

TaskDTO was created to carry task information between the API and the application. The API sends useful task fields without exposing the User entity directly.

6. Service layer – the main business logic

TaskService contains the actual task operations: create, get all, get one, update and delete.

Important security logic: before working with a task, the service finds the logged-in user using the email from the JWT. It then searches for the task using both the task ID and that user.

Example: User 1 owns Task 1. If User 2 asks for Task 1, the application does not find a Task 1 belonging to User 2.

7. Controller – API endpoints

Method	Endpoint	Purpose
POST	/tasks	Create a new task
GET	/tasks	Get the logged-in user's tasks
GET	/tasks/{id}	Get one task
PUT	/tasks/{id}	Update a task
DELETE	/tasks/{id}	Delete a task

8. JWT authentication – simple explanation

JWT is being used like a temporary digital pass. After successful login, the server creates a JWT containing the user's email. The client sends that token with requests that require login.

Flow:

1. User sends email + password.
2. Server checks the password.
3. Server creates a JWT.
4. Postman sends the JWT as a Bearer token.
5. JWT filter validates the token.
6. The application knows which user is making the request.

Example header:

```
Authorization: Bearer <JWT_TOKEN>
```

9. What we tested in Postman

Test	Result
Login and receive JWT	200 OK – JWT received
Create task	201 Created
Get all tasks	200 OK
Get task by ID	200 OK
Update task	200 OK
Delete task	204 No Content
User 2 security/isolation test	Access to another user's task was blocked

10. Real example from today's testing

We created a task named **“Learn Spring Boot”** with status **TODO** and due date **2026-10-10**. We successfully created it, retrieved it, updated it, and deleted it.

We also created a second user and tested access using the second user's JWT. The second user could not access the first user's task. This confirms that the ownership check is being applied.

11. What is still pending

Day 3 is mostly complete. The next improvements are:

- Proper error handling – for example, return 404 Not Found instead of a generic 500 when a task does not exist.
- Input validation – for example, prevent an empty task title and validate required fields.

After these are completed, Day 3 can be treated as fully finished. Pagination and filtering will be handled in a later step.

12. Day 3 in one sentence

We built the Task part of the Task Manager, connected it to users, protected it with JWT authentication, and successfully tested the complete CRUD flow in Postman.

Project: Task_Manager | Technology: Java 21, Spring Boot, Spring Security, JWT, Spring Data JPA, MySQL, Maven